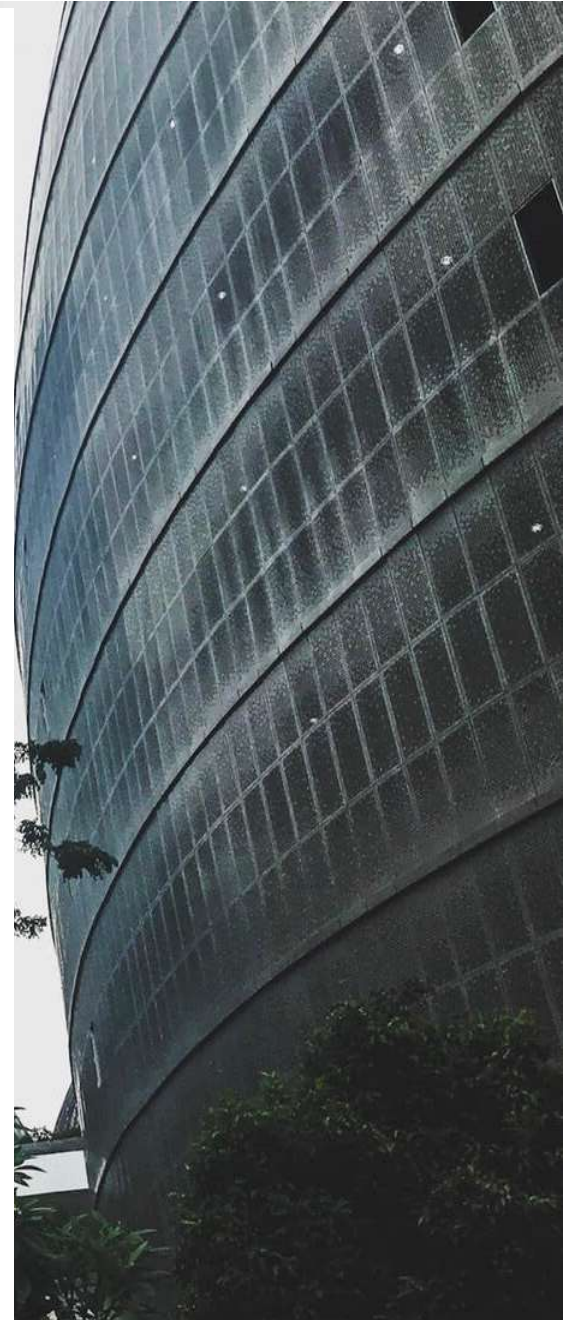


COURSE MATERIAL

**IF670 – Mobile Cross Platform Programming
S1 INFORMATICS GRADUATE PROGRAM
FACULTY OF ENGINEERING AND INFORMATICS**

**INFORMATICS STUDY PROGRAM
FACULTY OF ENGINEERING AND INFORMATION
UNIVERSITAS MULTIMEDIA NUSANTARA**

Gedung B Lantai 5, Kampus UMN
Jl. Scientia Boulevard, Gading Serpong, Tangerang, Banten-15811 Indonesia
Telp: +62-21.5422.0808 (ext. 1803), email: lab.fti@umn.ac.id, web: umn.ac.id



MODUL 8 (HTTP Request)

THEME DESCRIPTION

Students will learn how to handle HTTP requests using Axios.

WEEKLY LEARNING GOAL (SUB-GOAL LEARNING)

Students will learn how to handle HTTP requests using Axios and understand how to use it to communicate with APIs in application development.

SOFTWARE / HARDWARE REQUIREMENTS

1. Laptop / PC
2. IDE (Android Studio/XCode)
3. Code Editor (Visual Studio Code/Sublime/etc)

PRACTICUM STEPS

1. Create a new project and install **axios** by running the following command:
"npm i axios"
2. Create a file named **.env.local** in the root project directory to store your environment variables. Modify the file with the following configuration.

```
.env.local
1 EXPO_PUBLIC_API_URL = "https://jsonplaceholder.typicode.com/"
```

3. After installation, create a folder named **services** and create a file called **api.ts** inside it. Then, fill **api.ts** with the following code.

```
import axios from "axios";

const ENV = process.env.EXPO_PUBLIC_API_URL;

export const getPosts = () => {
  return axios.get(ENV+"posts");
}
```

4. Modify your **index.tsx** to use the functions you have created in **api.ts**.
 In this step, you will create a page that displays posts from all users (**posts**).
 When a post is clicked, it should navigate to the **Post Detail** page.
 Check your console to see the response!

```
import { router } from "expo-router";
import { useEffect, useState } from "react";
import { Pressable, ScrollView, Text, View } from "react-native";
import { getPosts } from "../services/api";

export default function Index() {
  const [posts, setPosts] = useState<any[]>([]);

  useEffect(() => {
    getAllPosts();
  }, []);

  const getAllPosts = () => {
    getPosts().then((res) => {
      if (res.status === 200) {
        setPosts(res.data);
        //check the data model from the response on the console log or network tab in the browser or the docs on the api link
        console.log(res.data);
      } else {
        console.log("error");
      }
    });
  };
}
```

```
25   return (
26     <View
27       style={{
28         flex: 1,
29         justifyContent: "center",
30         alignItems: "center",
31       }}
32     >
33       <ScrollView>
34         {posts.map((post) => (
35           <Pressable
36             key={post.id}
37             style={{padding: 10, borderWidth: 1}}
38             onPress={() => router.push({
39               pathname: "/postDetail",
40               params: {
41                 id: post.id,
42                 userId: post.userId
43               }
44             })}
45           >
46             <Text>Post Number: {post.id}</Text>
47             <Text>Title: {post.title}</Text>
48             <Text>Body: {post.body}</Text>
49           </Pressable>
50         ))}
51       </ScrollView>
52     </View>
53   );
54 }
55
```

5. In this step, you will perform **GET Post Detail** and **GET User** who created the post. Create a new file named **postDetail.tsx** with the following content:

```

1  import { getPostDetail, getUserDetail } from "@services/api";
2  import { useLocalSearchParams } from "expo-router";
3  import { useEffect, useState } from "react";
4  import { Text, View } from "react-native";
5
6  export default function PostDetail() {
7    const { id } = useLocalSearchParams<{ id: string }>();
8    const { userId } = useLocalSearchParams<{ userId: string }>();
9
10   const [user, setUser] = useState<any>(null);
11   const [post, setPost] = useState<any>(null);
12
13   useEffect(() => {
14     if (id) {
15       getPostDetailData();
16       getUserData();
17     }
18   }, []);
19
20   const getUserData = () => {
21     getUserDetail(Number(userId)).then((res) => {
22       if (res.status === 200) {
23         setUser(res.data);
24         console.log(res.data);
25       } else {
26         console.log("error");
27       }
28     });
29   };
30
31   const getPostDetailData = () => {
32     getPostDetail(Number(id)).then((res) => {
33       if (res.status === 200) {
34         setPost(res.data);
35         console.log(res.data);
36       } else {
37         console.log("error");
38       }
39     });
40   };
41
42   return (
43     <View
44       style={{
45         flex: 1,
46         justifyContent: "center",
47         alignItems: "center",
48       }}
49     >
50       <Text style={{ textAlign: "center", fontWeight: "bold" }}>{post?.title}</Text>
51       <Text style={{ textAlign: "center" }}>{post?.body}</Text>
52       <br></br>
53       <Text>Post Created By</Text>
54       <Text>Name: {user?.name}</Text>
55       <Text>Email: {user?.email}</Text>
56     </View>
57   );
58 }

```

6. Then, add functions in `api.ts` to retrieve Post Details and User who created the posts

```
export const getPostDetail = (id:number) => {
  return axios.get(ENV+"posts/"+id);
}

export const getUserDetail = (id:number) => {
  return axios.get(ENV+"users/"+id);
}
```

7. Below is a new function that can be added to `api.ts` to create a new post using the **POST** method!

```
export const postData = (data: {
  title: string,
  body: string,
  userId: number
}) => {
  return axios.post(ENV+"posts",data);
}
```

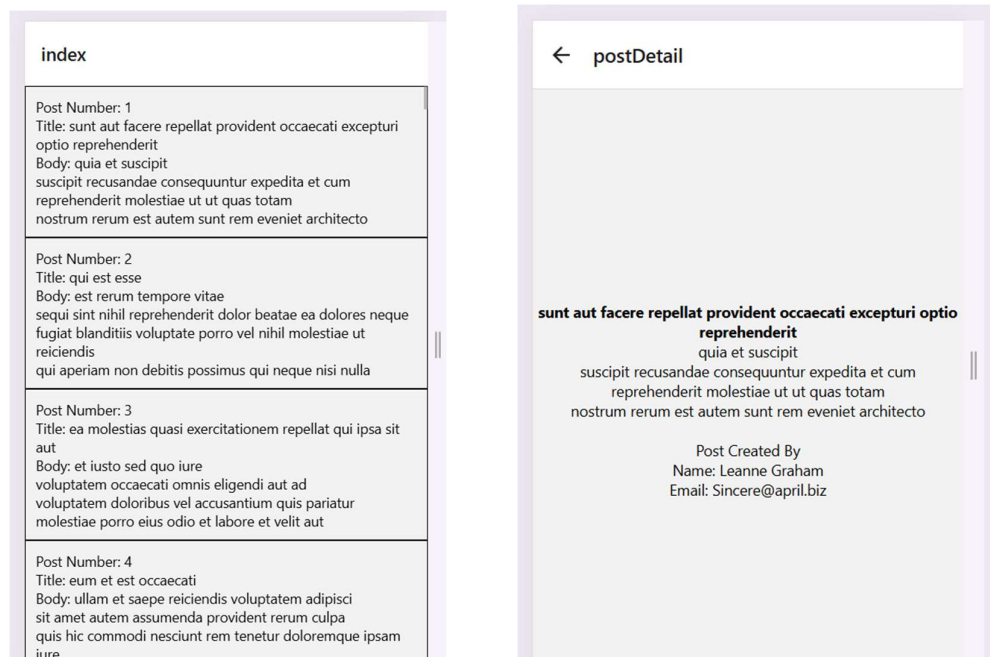
ASSIGNMENT

1. Create an "Add New Post" button on the main page.

You must implement the POST method using the `postData` function that you added in `api.ts`.

Design it creatively (e.g., button → navigate to a form page with Submit & Cancel buttons).

2. Display a list of comments on the Post Detail page. (Please refer to the documentation in the link below.)



REFERENCE

<https://jsonplaceholder.typicode.com/>